MONITOR Z · L01.4

Clase 4 — Los datos: DS, DC, literales, el PARM y el EX

Cuaderno de estudio · versión del 2026-10-28

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

L01.4 — Clase 4 · Los datos: DS, DC, literales, el PARM y el `EX`

Temario de la L01: 1. La forma de un programa · 2. Los registros y las instrucciones de todos los días · 3. Entrar y salir: prólogo, save area, epílogo y RC · **4. Los datos: DS, DC, literales, el PARM y el `EX`** · 5. Hablar con el sistema y leer el listado: macros, SVC, DCB, AMODE/RMODE.

Objetivo de esta clase: entender los **477 bytes de datos** de ZTKHOLA y las dos piezas más finas del programa: cómo llega el PARM y cómo el `EX` copia un texto de largo variable. Al terminar tenés que poder:

- leer una definición de datos (`DS`, `DC`, tipo, largo, duplicación) y decir cuántos bytes ocupa y dónde cae;
- explicar por qué a veces el `Loc` salta bytes (la alineación);
- decir qué es un literal y dónde lo pone `LTORG`;
- seguir el PARM desde el JCL hasta R3, y el `EX` hasta los bytes que copia.

> 📚 Fuentes: - `monitor_z/asm/ZTKHOLA.asm`, renglones 40–62 y 79–97. - El listado de **JOB24962**: sentencias 111–169 y la *Ordinary Symbol and Literal Cross Reference* (página 7). - JESJCL y RUN1/SYSPRINT de JOB24962: `PARM='PRUEBA'` y `+HOLA PARM=PRUEBA`. - *HLASM Language Reference* (SC26-4940): DS/DC, literales, LTORG. *Principles of Operation*, cap. 7: `EXECUTE`.

Respuestas de la clase 3

1. Con `LM 14,12,12(13)`, R15 volvería a tener lo que traía al entrar (la dirección de entrada de ZTKHOLA) y el RC 4 se perdería. Qué RC anotaría z/OS **no está medido**: sería el valor de esa dirección, no un 4.
2. Sin el `ST 13,4(,2)`, la palabra +4 de tu `SAVEAREA` queda con lo que haya en un `DS` sin valor. `L 13,4(,13)` cargaría eso en R13, y `L 14,12(,13)` y `LM 0,12,20(13)` leerían de una dirección cualquiera. Lo más probable es un ABEND al leer o al volver; **cuál, no está medido**.
3. Con ASM en 8, `COND=(4,LT,ASM)` se cumple en LKED, RUN1 y RUN2 (4 < 8): no corre ninguno. El `$HASP395` diría `RC=0008`, porque el más alto de los pasos que corrieron sería el de ASM.

1. DS y DC: reservar lugar, con o sin valor

Las dos directivas de datos tienen la misma forma: `nombre DS|DC [duplicación] tipo [L largo] ['valor']`.

Directiva	Qué hace	En ZTKHOLA
`DS` (*define storage*)	**reserva** lugar, sin valor inicial	`SALVAR1 DS F` · `SAVEAREA DS 18F` · `SALIDA DS CL133`
`DC` (*define constant*)	reserva lugar **y** le pone un valor	`ESPACIOS DC CL133' '`

Los tipos que aparecen en tu programa:

Tipo	Qué es	Largo por omisión	Se alinea a	Ejemplo
`F`	fullword, entero binario	4	múltiplo de 4	`RCFINAL DS F`
`H`	halfword, entero binario	2	múltiplo de 2	`=H'100'`
`C`	caracteres (EBCDIC)	1 (o el del valor)	nada	`CL133' '`
`A` / `AL3`	una **dirección** (4 o 3 bytes)	4	múltiplo de 4 (`AL3`: nada)	`AL3(PRTDCB)` en el `OPEN`
`B` / `BL1`	bits	1	nada	`BL1'10010000'` en el DCB

La duplicación va adelante: `18F` = 18 fullwords = 72 bytes. **El largo** va con `L`: `CL133` = 133 caracteres. **El truco del cero:** `DS 0H` reserva **cero** halfwords. No ocupa nada, pero **alinea** y le pone nombre a ese lugar. Así son tus etiquetas `ABRIO`, `LARGOOK`, `SINPARM` y `SALIR`: en la *Cross Reference* tienen tipo `H` y ningún byte propio.

`ESPACIOS DC CL133' '` se volvió `4040404040404040…` en el listado. `X'40'` es el **blanco en EBCDIC**, no el `X'20'` de ASCII. El listado muestra sólo los primeros 8 bytes de los 133.

DS y DC: reservar lugar, con o sin valor

L01 · Clase 4 · Los datos

```
nombre  DS|DC  [duplicación]tipo[L largo]['valor']
```

Tipo	Qué es	Largo	Se alinea a	En ZTKHOLA
`F`	fullword binario	4	múltiplo de 4	`RCFINAL DS F`
`H`	halfword binario	2	múltiplo de 2	`=H'100'` → `0064`
`C`	caracteres EBCDIC	1 o el del valor	—	`ESPACIOS DC CL133' '` → `4040…`
`A` / `AL3`	una dirección	4 / 3	4 / —	`AL3(PRTDCB)` → `00022C`
`B`	bits	1	—	`BL1'10010000'` (RECFM del DCB)

Duplicación

`18F` = 18 fullwords = **72 bytes**.

El truco del cero

`ABRIO DS 0H`: cero bytes. Alinea y pone nombre: para eso están tus etiquetas.

EBCDIC

El blanco es `X'40'`, no el `X'20'` de ASCII. El `+` es `X'4E'`.

Fuente: ZTKHOLA.asm (renglones 33, 83–87) · listado de JOB24962 (sentencias 115, 140, 168; página 7) · HLASM Language Reference

Lámina L01-22_ds_y_dc

2. El mapa de los datos, byte por byte

Con la columna `Loc` y la *Cross Reference* se arma el mapa completo de la quinta parte del programa:

Loc	Hasta	Bytes	Nombre	Definición	Tipo en la XREF
`0000CC`	`0000CF`	4	`SALVAR1`	`DS F`	F
`0000D0`	`0000D3`	4	`RCFINAL`	`DS F`	F
`0000D4`	`00011B`	72	`SAVEAREA`	`DS 18F`	F
`00011C`	`0001A0`	133	`SALIDA`	`DS CL133`	C
`0001A1`	`000225`	133	`ESPACIOS`	`DC CL133' '`	C
`000226`	`00022B`	6	`COPIAPRM`	`MVC` modelo	I (instrucción)
`00022C`	`00028B`	96	`PRTDCB`	macro `DCB`	F
`00028C`	`00028F`	**4**	—	**hueco**	—
`000290`	`0002A8`	25	literales	`LTORG`	F, F, F, H, C
		477			

Dos cosas que muestra el mapa:

- `SALVAR1` cae justo en `0CC` porque el `BR 14` terminó en `0CB` y `0CC` ya es múltiplo de 4. Si no lo fuera, el ensamblador habría saltado bytes hasta el próximo múltiplo: eso es la **alineación**. La CPU lee un `F` en cualquier dirección, pero las instrucciones como `L` y `ST` están pensadas para fullwords alineados, y las macros de IBM (como `DCB`, que arranca con `DC 0F'0'`) lo exigen.
- **El hueco de 4 bytes** (`28C`–`28F`): el DCB termina en `28B` y los literales arrancan en `290`. `LTORG` alinea el bloque de literales a **doble palabra** (múltiplo de 8): `28C` no lo es, `290` sí.



Lámina L01-23_mapa_de_datos

3. Los literales y `LTORG`

Un **literal** es una constante que escribís **directamente en el operando**, con un `=` adelante, sin definirla aparte:

```
         MVC   RCFINAL,=F'8'          ← el ensamblador crea un F con va
```

El ensamblador junta todos los literales en un **pool** y los pone donde está el `LTORG` (o, si no hay, al final del programa). Los tuyos, de la sentencia 164 en adelante:

Loc	Bytes	Literal	Lo usan las sentencias
`000290`	`00000008`	`=F'8'`	20 (RC por omisión)

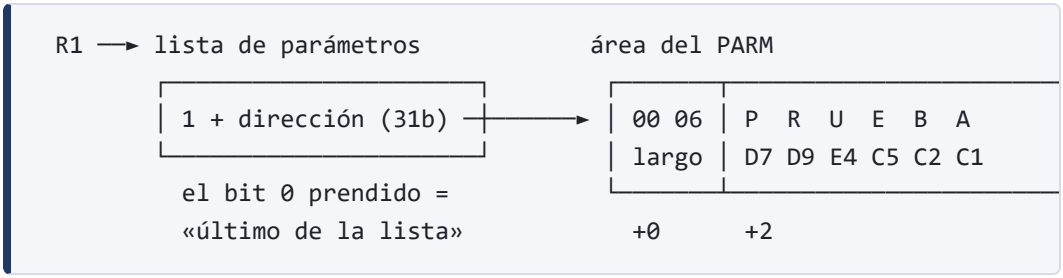
`000294`	`0000000C`	`=F'12'`	45 (error al abrir)
`000298`	`00000004`	`=F'4'`	84 (vino PARM)
`00029C`	`0064`	`=H'100'`	69 **y** 71: dos usos, **un solo** literal
`00029E`	`4EC8D6D3C140D7C1…`	`=C'+HOLA PARM='`	68

- El orden no es el del fuente: el ensamblador pone **primero los de 4 bytes, después el de 2 y al final el de caracteres**. Así no deja huecos entre ellos.
- Un literal repetido se guarda **una sola vez**: `=H'100'` aparece en 69 y 71 con la misma dirección `29C`.
- `4E` es el `+` en EBCDIC, `C8` la H, `D6` la O… El texto llega hasta `2A8`, el último byte del programa: `X'2A9'` **= 681**.

> ⚠️ **No modifiques un literal.** `MVC RCFINAL,=F'8'` copia **desde** el literal. Si alguna instrucción escribiera **en** `=F'8'`, cambiaría el 8 para todas las que lo usan.

4. El PARM: de `PARM='PRUEBA'` a R3

El JCL de RUN1 dice `PARM='PRUEBA'`. z/OS no te da el texto en un registro: te da una **cadena de direcciones**. Es lo que lee el bloque 2 (renglones 40–42):



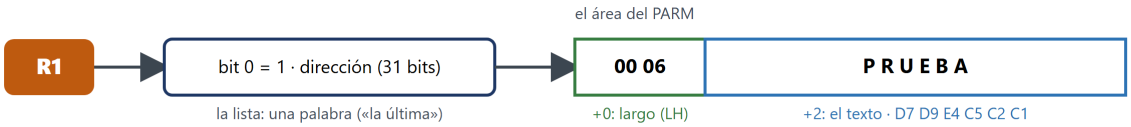
Renglón	Sentencia	Qué deja	En RUN1
15	`ST 1,SALVAR1`	la dirección de la lista, a salvo en memoria	—
40	`L 2,SALVAR1`	R2 = dirección de la lista	—
41	`L 2,0(,2)`	R2 = la primera palabra de la lista = dirección del área del PARM	—

42	`LH 3,0(,2)`	R3 = los 2 primeros bytes del área = **el largo**	6
—	`2(,2)`	el texto empieza 2 bytes después del largo	`PRUEBA`

- **El bit de arriba.** En la lista, la última dirección lleva prendido el bit 0 («ya no hay más»). En `AMODE 31` la CPU usa sólo los 31 bits de abajo para direccionar, así que `LH 3,0(,2)` funciona igual con el bit prendido.
- **Sin PARM** (RUN2) el área existe igual, con largo **0**. Por eso el `LTR 3,3` de la clase 2 da CC 0 y RUN2 sale con 8.
- El dibujo del área es **la convención de z/OS**, con los valores que corresponden a `PRUEBA` en EBCDIC. **No se volcó la memoria** de RUN1: lo que sí está medido es que el texto llegó, porque salió en el SYSPRINT.

El PARM: de PARM='PRUEBA' a R3

L01 · Clase 4 · Los datos



convención de z/OS con los valores de PRUEBA en EBCDIC · la memoria de RUN1 no se volcó

Renglón	Sentencia	Qué deja	RUN1
15	`ST 1,SALVAR1`	la dirección de la lista, a salvo	—
40	`L 2,SALVAR1`	R2 = la lista	—
41	`L 2,0(,2)`	R2 = el área del PARM (el bit 0 no molesta en AMODE 31)	—
42	`LH 3,0(,2)`	R3 = el largo	**6**
—	`2(,2)`	el texto, 2 bytes después	`PRUEBA`

> Sin PARM (RUN2) el área existe igual, con largo **0**: por eso `LTR 3,3` da CC 0 y sale con RC 8.

Fuente: ZTKHOLA.asm (renglones 15, 40–42) · JESJCL de JOB24962 (RUN1 PARM='PRUEBA') · MVS Assembler Services Guide

Lámina L01-24_el_parm

5. El `EX`: un `MVC` con el largo puesto en el momento

`MVC` lleva el largo **dentro de la instrucción** (clase 2), fijo al ensamblar. Pero el largo del PARM recién se sabe al correr. La solución es **`EX`** (*execute*): ejecuta **otra** instrucción, pero antes le hace un **OR** entre su segundo byte y el byte de abajo de un registro.

La instrucción modelo está **entre los datos** (renglón 93), después del `BR 14`, para que nunca se ejecute sola:

```
000226 D200 C127 2002   121 COPIAPRM MVC   SALIDA+11(0),2(2)
```

Parte	Valor	Qué es
`D2`	MVC	
`00`	largo guardado **0**	el que le ponga el `EX`
`C127`	base R12 + `127`	`SALIDA+11` (`11C + B = 127`): justo después de `+HOLA PARM=`
`2002`	base R2 + 2	el texto del PARM

En RUN1:

Renglón	Sentencia	R3	Qué pasa
42	`LH 3,0(,2)`	6	el largo del PARM
55–57	`CH 3,=H'100'` / `BNH` / `LH 3,=H'100'`	6	no pasa de 100, así que no se toca
59	`BCTR 3,0`	5	largo − 1, porque `MVC` cuenta desde 0
60	`EX 3,COPIAPRM` (`4430 C226`)	5	se ejecuta `D2 05 C127 2002`: copia **6** bytes
61	`PUT PRTDCB,SALIDA`	—	escribe `+HOLA PARM=PRUEBA` ✅ medido en el SYSPRINT de RUN1

Las dos protecciones que hay antes del `EX`:

Protección	Renglones	Qué evita
`LTR 3,3` + `BZ SINPARM`	43–44	Con R3 = 0, `BCTR` daría **−1** (`X'…FF'`). El `EX` pondría `FF` = copiar **256** bytes: se pasaría de `SALIDA` y pisaría `ESPACIOS`.
`CH 3,=H'100'` + tope	55–57	11 bytes de título + 100 de texto = 111, que entran en los 133 de `SALIDA`. Un largo que viene de afuera nunca se usa sin tope.

EX: un MVC con el largo puesto en el momento

La modelo (renglón 93, entre los datos)

`D2 00 C127 2002`

MVC · largo **0** · **SALIDA+11** · **R2 + 2** (el texto)

→ `EX 3` →

Lo que se ejecuta en RUN1 (R3 = 5)

`D2 05 C127 2002`

OR del byte de abajo de R3 con el segundo byte: copia **6** bytes

Renglón	Sentencia	R3	Qué pasa
42	`LH 3,0(,2)`	6	el largo del PARM
55–57	`CH` · `BNH` · tope	6	no pasa de 100
59	`BCTR 3,0`	5	largo − 1
60	`EX 3,COPIAPRM` · `4430 C226`	5	`D205…` copia `PRUEBA`
61	`PUT PRTDCB,SALIDA`	—	✅ `+HOLA PARM=PRUEBA` **en SYSPRINT**

⚠️ **Las dos protecciones.** Con R3 = 0, `BCTR` daría −1 → `FF` → ¡256 bytes! Lo evita `LTR` / `BZ` (43–44). Y el tope de 100 (55–57) garantiza que 11 + texto entra en los 133 de `SALIDA`.

Fuente: listado de JOB24962 (sentencias 56–74, 121) · RUN1/SYSPRINT de JOB24962 · Principles of Operation, cap. 7 (EXECUTE)

Lámina L01-25_el_ex

Resumen

Idea	En una línea	Testigo
`DS` / `DC`	reservar lugar; `DC` además le pone valor	renglones 83–87
Tipo, duplicación, largo	`18F` = 72 bytes; `CL133` = 133; `DS 0H` = 0 bytes, sólo alinea y pone nombre	*Cross Reference*, página 7
Alineación	`F` a múltiplo de 4, `H` a 2, `LTORG` a 8: por eso el hueco de 4 bytes	`28C – 28F`
Literales	`=F'8'` sin definirlo aparte; van al pool del `LTORG`, ordenados por largo, sin repetidos	sentencias 165–169
EBCDIC	blanco = `X'40'`; `+` = `X'4E'`	`ESPACIOS`, `=C'+HOLA PARM='`
El PARM	R1 → lista → área: 2 bytes de largo + el texto	renglones 40–42; RUN1 y RUN2
`EX`	ejecuta el `MVC` modelo con el largo de R3 (OR en el segundo byte)	`D200` → `D205` en RUN1

Protecciones	`LTR` / `BZ` (largo 0 → 256 bytes) y tope 100	renglones 43–44 y 55–57

Preguntas de comprobación

1. Mirá el mapa de la sección 2. Si agregaras `CONTADOR DS H` justo después de `RCFINAL`, ¿en qué `Loc` caería `SAVEAREA`? ¿Cuántos bytes de hueco aparecerían, y por qué?
2. Si borraras el renglón 96 (`LTORG`), ¿dónde quedarían los literales? ¿Cambiaría el `Loc` de algo más?
3. Con `PARM='A'` el `EX` ejecuta `D2?? C127 2002`. ¿Qué byte va en `??`? ¿Y si el PARM tuviera 150 caracteres?

Mis respuestas

Relacionado: *L01.3 - Clase 3 - Entrar y salir - prologo, save area, epilogo y RC* · *L01.5 - Clase 5 - Hablar con el sistema y leer el listado - macros, SVC, DCB, AMODE y RMODE* · *P01 - ZTKHOLA* · *Ruta de aprendizaje*.

Notas